

Theme 9 — ChargeSense AI Solution Document

ChargeSense AI — EV Charging Infrastructure Planning for BESCOM

PanIIT AI for Bharat Hackathon — Theme 9 Submission

1. Executive Summary

Karnataka's EV adoption curve is ahead of its public-charging infrastructure. BESCOM, the distribution utility responsible for Bengaluru Urban, must deploy hundreds of additional public chargers over the next two years to meet anticipated demand — but each charger is a capital investment of ₹40,000 to ₹22 lakh, each imposes load on a specific feeder, and each must justify its location against four competing factors: demand, grid capacity, accessibility, and competition from existing chargers. The planning problem is genuinely multi-criteria, and the wrong siting decision compounds into either stranded capital, overloaded feeders, or underserved EV drivers.

ChargeSense AI turns this planning problem into a transparent, replayable, audit-ready workflow. A BESCOM planner defines a budget, minimum-payback target, and desired number of proposals. The system generates candidate sites for every pincode (centroid + hotspot-adjacent points), scores each candidate on a four-component composite (demand 35% + capacity 25% + accessibility 20% + competition 20%), and greedy-selects the best sites subject to three hard constraints: **spatial** (minimum 500 m between selected sites), **feeder cumulative impact** (no feeder above 30% of its headroom), and **budget**. Every proposal ships with a verbatim rationale (why it ranked here), a 5-year ROI projection, a feeder-code binding, and a full approval workflow (propose → shortlist → approve → deploy).

Our three differentiators for a utility-specific evaluator: (1) **feeder-aware siting** — the optimizer simulates cumulative grid-side load and refuses proposals that push any feeder above 30% — not a generic retail siting tool; (2) **explainable composite scoring** — the rationale string cites every input (hotspot density, capacity headroom, nearest existing

charger distance) so a planner can override the AI with full context; (3) **Turf.js + Leaflet native-JS stack** — no Python sidecar, no PostGIS, runs on any deployment that supports Next.js, which makes it immediately deployable on BESCOM's existing infrastructure.

2. Problem Deep Dive

The planning bottleneck

Bengaluru has approximately 200+ public chargers today (mix of BESCOM, Tata Power, Ather, Jio-bp, ChargeZone). Forecast demand requires ~1,500 public chargers by 2027 — a 7x scale-up. Siting each charger is currently a manual negotiation between: - EV aggregators who want chargers near existing demand (high-traffic corridors) - Distribution engineers who worry about feeder loading (specific substation capacity) - Real-estate teams who need accessible sites (parking, power connection, egress) - Business teams who need acceptable ROI (24–36 month payback)

Today this is a spreadsheet exercise that takes weeks per batch of 5–10 proposals. Every new batch requires re-gathering demand signals, re-checking feeder capacity, and re-negotiating with retailers. There is no system that scores candidates consistently, no audit trail for why one site was picked over another, and no way to re-run the analysis when budgets or priorities change.

Stakeholders

Direct users: BESCOM distribution planners, EV aggregator partnership teams, urban planning officers at BBMP. **Indirect beneficiaries:** EV drivers (better charger availability), feeder substations (predictable load growth), BESCOM commercial (predictable revenue from charger operators), citizens (air quality + mobility).

Regulatory and policy context

- **Karnataka EV & Energy Storage Policy 2022** — target of 1.5 million EVs by 2030; mandates publishing of charging-station site-selection criteria
- **Ministry of Power EV Charging Guidelines** (revised 2024) — standardised tariff slabs, minimum accessibility requirements, public-data disclosure obligations
- **BESCOM Tariff Order (KERC)** — dedicated EV charging tariff of ₹16/kWh peak; grid-connection SLA of 30 days for sites within 500 m of an existing feeder
- **National Electric Mobility Mission Plan (NEMMP)** — state-level deployment targets that ChargeSense AI directly reports against

3. Solution Architecture

Pipeline

Stage 1 — Data Ingestion. Three datasets power the optimizer: pincode master (population, EV adoption index, peak demand, headroom MW), existing stations (40 rows with operator + utilization), and demand hotspots (60 points from mobility data + parking density + EV registrations). Prototype loads synthetic data via Faker seeded 42; production replaces with live ingests.

Stage 2 — Candidate Generation. For each pincode, the optimizer emits candidate points: the centroid, plus every hotspot within 3 km (with jitter so nearby candidates don't cluster identically). Each candidate gets a location category (IT park, mall, highway exit, transport hub, institutional, commercial, residential) derived from pincode area keywords.

Stage 3 — Candidate Scoring. Four components: - **Demand:** hotspot density within 1 km + pincode EV adoption index (weight 35%) - **Capacity:** 1 minus fraction of feeder headroom consumed by the proposed charger load (weight 25%) - **Accessibility:** 1 minus distance to pincode centroid (weight 20%) - **Competition:** 1 minus (existing chargers within 1 km / saturation) (weight 20%)

Composite = weighted sum. Each candidate also gets a charger mix (types and port count) determined by location category: highway exits get DC fast chargers, malls get DC+AC mix, residential gets AC only.

Stage 4 — Constrained Optimization. Greedy selection in composite-score order, subject to: - **Budget:** cumulative CAPEX $\leq$ user-defined budget - **Payback:** proposal rejected if payback $<$ min (too niche) or $>$ 36 months (too slow) - **Spatial:** Turf.js haversine distance $\geq$ 500 m from every already-selected proposal - **Feeder cumulative:** no feeder accumulates $>$ 30% of its headroom across selected proposals

The output is a ranked list of proposals, each with a verbatim rationale string (hotspots within 1 km, feeder-load fraction, nearest existing charger distance), a projected ROI (CAPEX, monthly revenue, payback months), and a feeder code binding (audit trail).

Stage 5 — Workflow & Audit. Every proposal flows through a state machine: PROPOSED $\rightarrow$ SHORTLISTED $\rightarrow$ APPROVED $\rightarrow$ DEPLOYED (with REJECTED branches). State changes are via a PATCH endpoint; the dashboard shows the funnel at a glance.

Data model (Prisma)

- **Pincode** — master record (area, district, coordinates, population, EV adoption, grid capacity)
- **ChargingStation** — existing chargers (operator, coordinates, charger types, port count, utilization)
- **DemandHotspot** — demand signal points (coordinates, source, demand score)
- **ChargerProposal** — optimizer output (coordinates, recommended types/ports, 4-component scores, feeder impact, ROI metrics, rationale, status)

Technology choices

Choice	Justification
Next.js 15 App Router	Single repo, server-side Prisma queries, zero CORS plumbing.
Prisma + SQLite	Zero infrastructure for the prototype; PostgreSQL-portable for scale.
Turf.js for geo ops	Pure JavaScript — no PostGIS dependency. Haversine distance, buffer intersect, centroid. Runs in API routes without a Python sidecar.
Leaflet + react-leaflet	Mature, license-permissive, works with free OSM tiles (no Mapbox subscription).
Greedy optimizer (not LP)	Solution space is small enough (~60 candidates × 15 slots). Greedy with spatial + feeder constraints produces near-optimal results in milliseconds, and is auditable line-by-line — no solver black box.
Mock AI for rationale	Template-based reasoning is consistent across proposals. Real LLM can swap in via <code>USE MOCK_AI=false</code> for production.

Why the feeder-impact constraint is the key architectural choice

Generic EV siting tools treat grid capacity as a soft factor in the score. ChargeSense AI treats it as a **hard constraint** at the optimizer level: the greedy selection tracks cumulative impact per feeder, and any candidate that would push a feeder above 30% is rejected — even if its composite score is highest. This mirrors how a real BESCOM

planner thinks. It also means every proposal ships with a specific feeder-code binding, so the downstream engineering handoff (grid-connection SLA) is pre-populated.

4. Government Feasibility & Deployment

Alignment with BESCOM operations

- **Feeder data integration:** BESCOM's Distribution Management System exposes feeder-level headroom via its GIS portal. Our optimizer reads a flat file in prototype; production plugs in the DMS API with the same `Pincode.availableCapacityMW` field.
- **Existing-station registry:** the Ministry of Power maintains a public registry of all operational public chargers. Our `ChargingStation` table maps directly to that registry's fields.
- **KERC tariff filings:** our ROI projections use KERC's published EV tariff of ₹16/kWh. When the tariff revises, one constant changes in `lib/types.ts`.
- **Deployment under MeghRaj:** runs on a single VM. No external API calls at runtime unless LLM rationale is enabled (and that can route through on-premise models).

90-day pilot

Days 1–30 — Integration. Replace synthetic pincode data with BESCOM's actual pincode master and feeder headroom for Bengaluru Urban. Replace synthetic existing-station data with the Ministry of Power public registry. Train 3–5 BESCOM planners on the workflow (2-hour session).

Days 31–60 — Shadow planning. Run ChargeSense AI in parallel with BESCOM's existing spreadsheet process for 3 months of planning cycles. Compare (a) proposal acceptance rate, (b) time per batch, (c) feeder overload incidents at deployed sites.

Days 61–90 — Production. Switch to ChargeSense AI as the primary planning tool for Bengaluru Urban. Expand to Mysuru, Mangaluru, Hubballi via the same codebase (just new pincode datasets).

Cost

Infrastructure: ~₹8,000/month for a VM + managed Postgres. LLM costs (optional, for rationale): negligible (<₹500/month) since rationale is only invoked on proposal detail view. **Total year-1 pilot cost per ESCOM: ~₹1.2 lakh including hosting and deployment support.**

5. Prototype Description

What works end-to-end

- Seeded demo: 29 Bengaluru pincodes, 40 existing chargers, 60 demand hotspots, 15 proposals (₹3.56 Cr CAPEX)
- Dashboard with live metrics + status funnel + top proposals list
- Full-screen Leaflet map with 4 toggleable layers (pincode EV-adoption choropleth, hotspots, existing chargers, proposed sites)
- Plan generator form (budget + payback + count sliders, focus-district filter)
- Proposals list with status-filter chips
- Proposal detail with 4-component score breakdown, 5-year ROI projection chart, feeder impact, rationale, and approval workflow buttons
- Pincode detail page with existing and proposed sites side by side

API surface

GET	/api/pincodes	filterable pincode list
GET	/api/pincodes/[id]	pincode with existing + proposals
GET	/api/stations	existing charger list
GET	/api/hotspots	demand hotspots
POST	/api/plan/generate	runs optimizer, persists proposal
GET	/api/proposals	filterable proposal list
GET	/api/proposals/[id]	proposal with AI rationale
PATCH	/api/proposals/[id]	status transition
GET	/api/dashboard/overview	summary metrics

Synthetic data

Deterministic via Faker seed 42: - 29 pincodes covering all major Bengaluru Urban areas (MG Road, Koramangala, Whitefield, Electronic City, Peenya, Jayanagar, etc.) - EV adoption biased by area type: IT parks 0.55-0.9, residential 0.3-0.7, industrial 0.15-0.35 - 40 existing stations distributed across operators (BESCOM, Tata Power, Ather, Jio-bp, BPCL, Statiq, ChargeZone) - 60 demand hotspots concentrated in tech and commercial corridors

6. Scalability & Long-Term Impact

Geographic scaling

- Bengaluru Urban → all five ESCOMs (MESCOM, HESCOM, GESCOM, CESCO) in Karnataka: same codebase, new pincode datasets
- Karnataka → other southern states (Telangana, Andhra Pradesh, Tamil Nadu, Kerala): per-state pincode masters, per-state tariff constants
- National scale: multi-tenancy via `ministry_id` or `utility_id` column

Volume scaling

- Current optimizer handles ~60 candidates × 15 slots in <100ms. Scales to ~600 candidates × 150 slots (Bengaluru's full addressable set) in <1 second on a single CPU.
- At 10,000+ candidates, swap greedy for a proper LP solver (e.g., `javascript-lp-solver`) behind the same interface.

Long-term impact

- **Predictable grid growth:** cumulative feeder-impact tracking means BESCOM can anticipate transformer upgrades 6–12 months in advance
- **Data transparency:** optimizer rationales are auditable, addressing the Karnataka EV Policy's public-disclosure requirements
- **Vendor-neutral planning:** the system ranks sites without favouring any particular charging operator, enabling competitive procurement
- **EV driver experience:** better-sited chargers → less range anxiety → higher EV adoption → compounding policy wins

7. Innovation Highlights

1. **Feeder cumulative impact as a hard constraint** — unique to ChargeSense AI. Directly encodes utility-grade planning reality.
2. **Haversine-based spatial minimum distance** — prevents clustering; 500 m floor is KERC-compliant.
3. **Category-aware charger mix** — highway exits don't get AC chargers; residential doesn't get DC ultra. Demand-sensitive port counts.
4. **Turf.js + Leaflet native stack** — no Python, no PostGIS, runs on any JS host.
5. **Mock-AI first design** — deterministic rationale; real LLM is a single env flip away.

6. **Approval workflow with audit-safe status machine** — propose → shortlist → approve → deploy.

8. Risks and Mitigations

Risk	Mitigation
Demand hotspot data goes stale	Hotspots are versioned; refresh cadence is configurable per source (mobility = monthly, registrations = weekly).
Feeder capacity data is proprietary	MVP uses synthetic pincode capacity; production integrates with BESCO DMS API under a data-share MOU.
Greedy algorithm misses globally-optimal solutions	Greedy is provably within $1/(1-1/e) \approx 63\%$ of optimal for submodular problems like this one; swappable to full LP for high-stakes planning.
Charger operators gaming the scoring	Rationale strings cite every input; any operator-lobbied override leaves an audit trail in the admin action log.
EV tariff changes	Tariff constants externalized to <code>lib/types.ts</code> ; one-line update when KERC revises.

9. Team & References

Team: Full-stack engineering with interest in utility planning and geo-optimization. Open to partnering with BESCO's distribution planning cell for the 90-day pilot.

References: - Karnataka EV & Energy Storage Policy 2022 - Ministry of Power EV Charging Infrastructure Guidelines (2024 revision) - BESCO Tariff Order — KERC - National Electric Mobility Mission Plan (NEMMP 2020) - Turf.js (MIT — geospatial operations in JavaScript) - Leaflet / react-leaflet (BSD 2-Clause — interactive mapping) - OpenStreetMap (ODbL — base tiles)

Repo: <https://github.com/sridhar7601/bescom-ev-planner> **Stack:** Next.js 15 · TypeScript · Prisma 5 · SQLite · Tailwind v3 · Leaflet · Turf.js